

Lenguajes Formales: características, descripciones, operaciones, usos [G9.5]. Autómatas finitos reconocedores [HMU2.1]. Autómatas finitos traductores [A 1]. Diagramas de Estado [HMU2.1]. Equivalencia entre AF deterministas y no deterministas [HMU2.2-2.3]. Minimización de AF [A1]. Expresiones regulares [HMU3]. Propiedades de lenguajes regulares [HMU4]. Gramáticas [G9.5]. Relación entre gramáticas regulares, lenguajes regulares y autómatas finitos [A1].

GERSTING, JUDITH L. “Mathematical Structures for Computer Science: A Modern Approach to Discrete Mathematics”. W H Freeman & Co, 2014.

HOPCROFT, JOHN E., MOTWANI, RAJEEV Y ULLMAN, JEFFREY D. “Introducción a la Teoría de Autómatas, Lenguajes y Computación”. Tercera Edición. Addison Wesley. 2006.

AUGUSTO, JUAN CARLOS . “Fundamentos de Ciencias de la Computación” – Notas de Curso. Universidad Nacional del Sur, Argentina. 2002.

Un **alfabeto** (o vocabulario)  $\Sigma$  es un conjunto finito no vacío de símbolos.

Una **cadena** (o palabra) sobre  $\Sigma$  es una secuencia finita de símbolos de  $\Sigma$ .

$\Sigma^*$  es el conjunto de todas las cadenas sobre  $\Sigma$ .

Un **lenguaje sobre  $\Sigma$**  es un subconjunto de  $\Sigma^*$ .

Los lenguajes son conjuntos (finitos o infinitos) de cadenas y por tal motivo podemos operar sobre ellos:

## Operaciones:

- unión
- intersección
- complemento
- concatenación
- estrella de Kleene

Sean  $L1$  y  $L2$  dos lenguajes sobre un alfabeto  $\Sigma$  :

$$L1 \cup L2 = \{w \mid w \in L1 \text{ ó } w \in L2\}$$

$$L1 \cap L2 = \{w \mid w \in L1 \text{ y } w \in L2\}$$

$$L1' = \{w \mid w \in \Sigma^* \text{ y } w \notin L1\}$$

$$L1 \circ L2 = \{w_1 w_2 \mid w_1 \in L1 \text{ y } w_2 \in L2\}$$

$$L1^* = \{w_1 w_2 \dots w_n \mid w_i \in L1, n \geq 0\}$$

## Ejercicios

Sea  $\Sigma = \{a,b\}$  y los siguientes lenguajes definidos sobre  $\Sigma$

Sea  $L1 = \{a^n b^n \mid n \geq 0\}$  y sea  $L2 = \{b^n \mid n \geq 0\}$

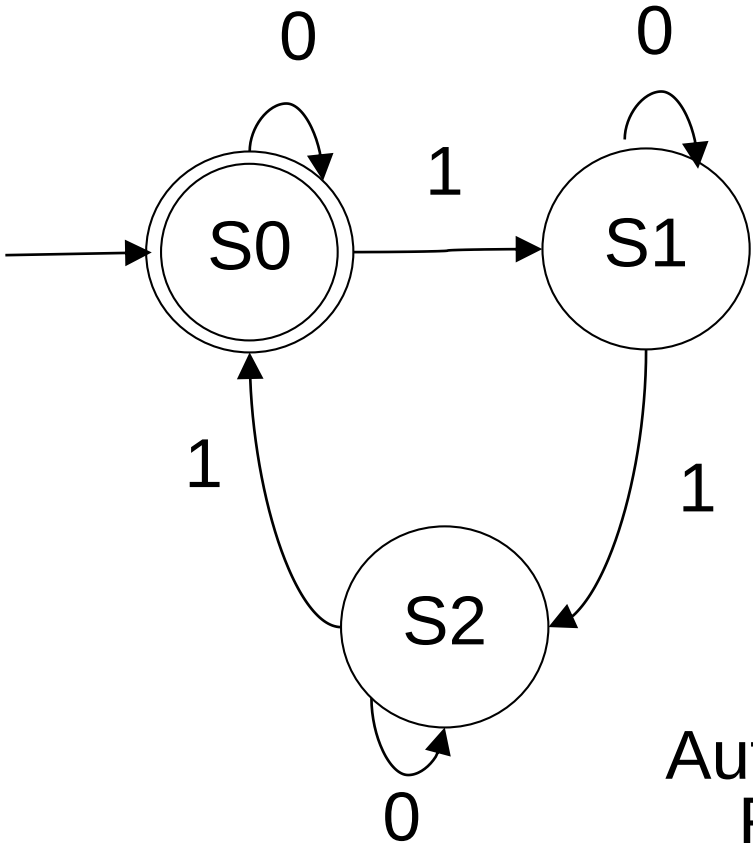
Identificar:

- $L1 \cup L2$
- $L1 \cap L2$
- $L1'$  y  $L2'$
- $L1 \circ L2$  y  $L2 \circ L1$
- $L1^*$  y  $L2^*$

**Ejemplo:**

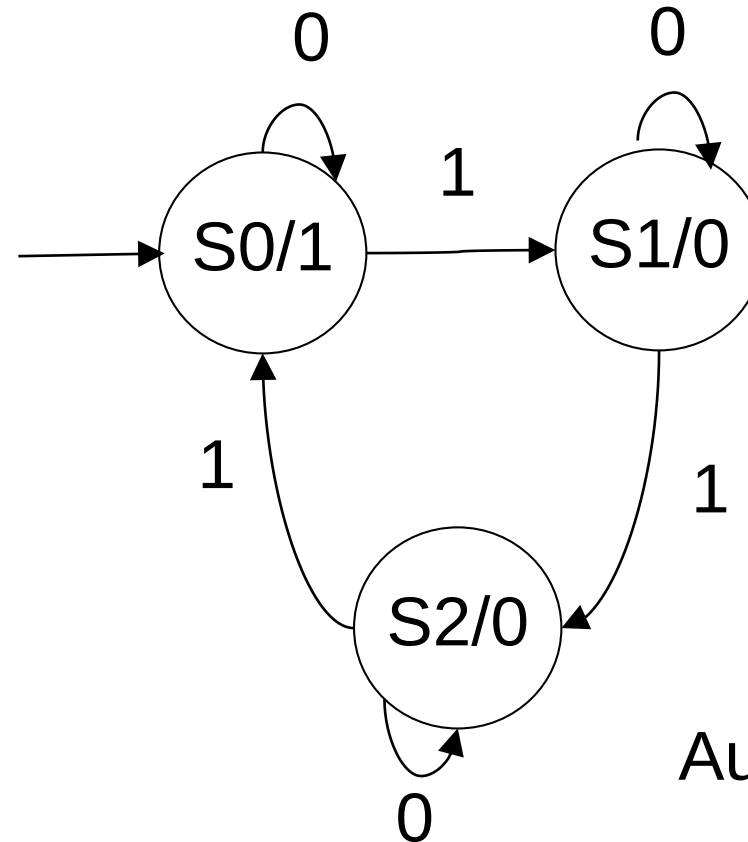
$$L1 \circ L2 = \{a^n b^m \mid m \geq n \geq 0\}$$

$$L2^* = L2$$



Autómata Finito  
Reconocedor

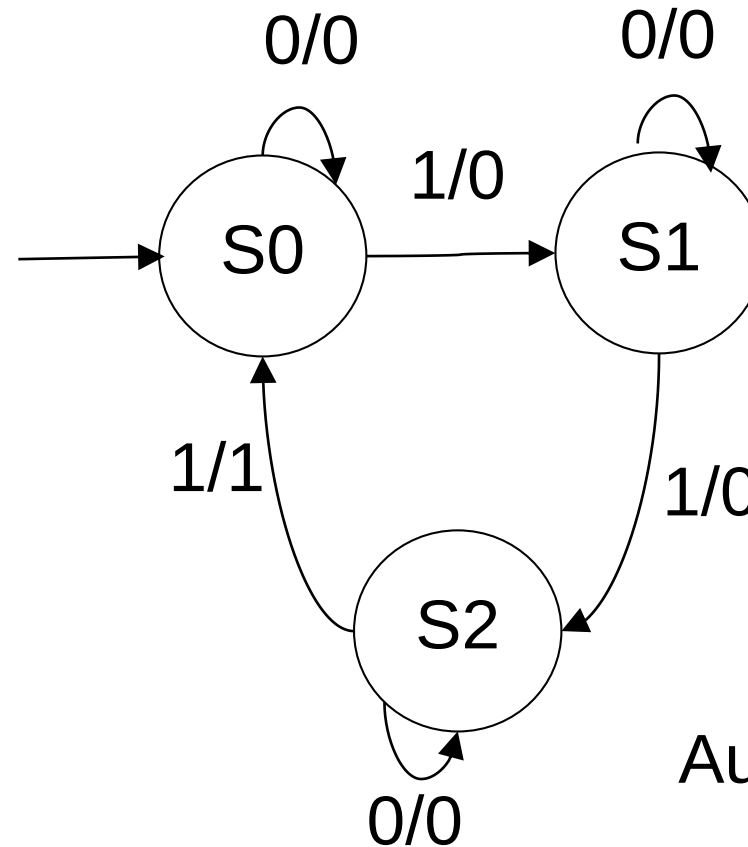
Reconoce el lenguaje sobre  $\Sigma=\{0,1\}$  con un número múltiplo de 3 de 1's



Autómata Finito  
De Moore

Emite 1 mientras el número de 1's leídos hasta el momento sea múltiplo de 3, de lo contrario emite 0





Autómata Finito  
De Mealy

Emite 1 cuando el número de 1's leídos se vuelve múltiplo de 3 ( $\neq 0$ ), de lo contrario emite 0

Un autómatata finito es un modelo que captura las características de una computadora.

Veremos dos tipos de autómatas finitos

- Autómatas Finitos Reconocedores (AFR)

Son capaces de reconocer lenguajes regulares.

- Autómatas Finitos Traductores

Toman una entrada y la traducen en una salida.

**Autómatas de Moore:** las salidas van asociadas a los estados

**Autómatas de Mealy:** las salidas van asociadas a las transiciones

Un *autómata finito reconocedor* (AFR) es una quintupla

$M = (S, \Sigma, \delta, s_0, F)$ , donde:

$S$  es un conjunto finito de estados,  $S \neq \emptyset$

$\Sigma$  es el alfabeto de entrada

$\delta: S \times \Sigma \rightarrow S$  es la función de transición

$s_0$  es el estado inicial,  $s_0 \in S$ .

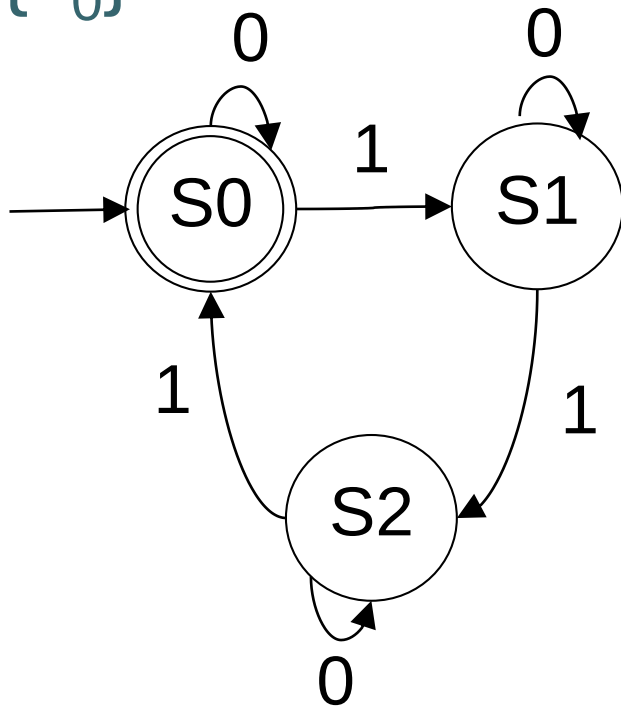
$F$  es el conjunto de estados finales o aceptadores ( $F \subseteq S$ )

## Ejemplo

$$M = (S, \Sigma, \delta, s_0, F),$$

$$S = \{s_0, s_1, s_2\}, \Sigma = \{0,1\}, F = \{s_0\}$$

| $\delta$ | 0     | 1     |
|----------|-------|-------|
| $s_0$    | $s_0$ | $s_1$ |
| $s_1$    | $s_1$ | $s_2$ |
| $s_2$    | $s_2$ | $s_0$ |



## Ejemplo

Dar un autómata finito reconocedor para el lenguaje de las cadenas definidas sobre el alfabeto  $\Sigma=\{a,b\}$  tales que empiezan con a.

$$M = (S, \Sigma, \delta, s_0, F),$$

$$S = \{s_0, s_1, s_2\},$$

$$\Sigma = \{a,b\},$$

$$F = \{s_1\}$$

| $\delta$ | a     | b     |
|----------|-------|-------|
| $s_0$    | $s_1$ | $s_2$ |
| $s_1$    | $s_1$ | $s_1$ |
| $s_2$    | $s_2$ | $s_2$ |

## Ejercicios

1. Dar un AFR para el lenguaje definido sobre el alfabeto  $\Sigma=\{0,1,2,3,4,5,6,7,8,9\}$  tal que reconozca los números pares.
2. Dar un AFR para el lenguaje definido sobre el alfabeto  $\Sigma=\{a,b\}$  cuyas cadenas tienen la letra b como segundo símbolo.
3. Dar un AFR para el lenguaje definido sobre el alfabeto  $\Sigma=\{a,b\}$  cuya longitud es par pero no divisible por 6.
4. Dar un AFR para el lenguaje definido sobre el alfabeto  $\Sigma=\{0,1\}$  tal que las cadenas tienen un número par de 0's y número par de 1's.

Definimos  $\delta^*: S \times \Sigma^* \rightarrow S$  como sigue:

$$\delta^*(q, \lambda) = q$$

$$\delta^*(q, aw) = \delta^*(\delta(q, a), w)$$

donde  $a \in \Sigma$  y  $w \in \Sigma^*$

Una forma equivalente de definir  $\delta^*$ :

$$\delta^*(q, \lambda) = q$$

$$\delta^*(q, wa) = \delta(\delta^*(q, w), a)$$

El *lenguaje reconocido* por el AFR

$M = (S, \Sigma, \delta, s_0, F)$ , se denota  $L(M)$  y se define como:

$$L(M) = \{w \mid \delta^*(s_0, w) \in F\}$$

Es decir, el lenguaje de  $M$  es el conjunto de cadenas  $w$  tales que comenzando del estado inicial de  $M$  nos llevan a algún estado final.

Si  $L = L(M)$  para algún  $M$ , entonces  $L$  es regular.



Un autómata finito traductor de Moore es una séxtupla

$$M = (S, \Sigma, \Gamma, \delta, s_0, f_0),$$

donde:

$S$  es un conjunto finito de estados,  $S \neq \emptyset$ .

$\Sigma$  es el alfabeto de entrada.

$\Gamma$  es el alfabeto de salida.

$\delta: S \times \Sigma \rightarrow S$  es la función de transición

$s_0$  es el estado inicial,  $s_0 \in S$ .

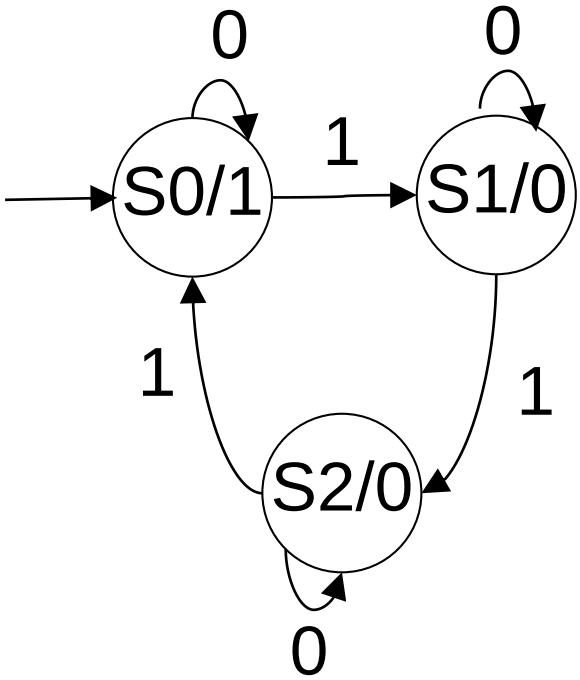
$f_0: S \rightarrow \Gamma^*$  es la función de salida.

## Ejemplo

$$M = (S, \Sigma, \Gamma, \delta, s_0, f_0),$$

$$S = \{s_0, s_1, s_2\}, \Sigma = \{0,1\}, \Gamma = \{0,1\}$$

| $\delta$ | 0     | 1     | $f_0$ |
|----------|-------|-------|-------|
| $s_0$    | $s_0$ | $s_1$ | 1     |
| $s_1$    | $s_1$ | $s_2$ | 0     |
| $s_2$    | $s_2$ | $s_0$ | 0     |



## Ejercicios

1. Dar un autómata finito traductor de Moore que emita un 1 cada vez que se haya leído la secuencia *bbba*.
2. Dar un autómata finito traductor de Moore que simule la función booleana AND definida sobre dos cadenas de la misma longitud. La salida para 11010 y 10010 debe ser 10010.
3. Dar un autómata finito traductor de Moore que simule la función booleana OR definida sobre dos cadenas de la misma longitud. La salida para 11010 y 10010 debe ser 11010.

Un autómata finito traductor de Mealy es una séxtupla

$$M = (S, \Sigma, \Gamma, \delta, s_0, f_0),$$

donde:

$S$  es un conjunto finito de estados,  $S \neq \emptyset$ .

$\Sigma$  es el alfabeto de entrada.

$\Gamma$  es el alfabeto de salida.

$\delta: S \times \Sigma \rightarrow S$  es la función de transición

$s_0$  es el estado inicial,  $s_0 \in S$ .

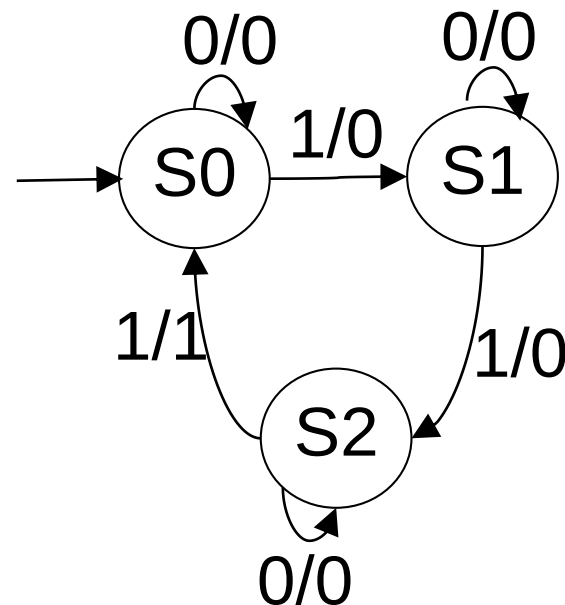
$f_0: S \times \Sigma \rightarrow \Gamma^*$  es la función de salida.

## Ejemplo

$$M = (S, \Sigma, \Gamma, \delta, s_0, f_0),$$

$$S = \{s_0, s_1, s_2\}, \Sigma = \{0,1\}, \Gamma = \{0,1\}$$

| $\delta / f_0$ | 0       | 1       |
|----------------|---------|---------|
| $s_0$          | $s_0/0$ | $s_1/0$ |
| $s_1$          | $s_1/0$ | $s_2/0$ |
| $s_2$          | $s_2/0$ | $s_0/1$ |



## Ejercicio

1. Convertir los autómatas de Moore de los ejercicios previos a autómatas de Mealy.
2. Dar un autómata finito traductor de Mealy que realice suma binaria. Indicar claramente cuáles son las convenciones adoptadas.

Un *autómata finito reconocedor no determinista* (AFRND) es una quintupla

$$M = (S, \Sigma, \delta, s_0, F),$$

donde:

$S$  es un **conjunto finito de estados**,  $S \neq \emptyset$ .

$\Sigma$  es el **alfabeto de entrada**.

$\delta: S \times \Sigma \rightarrow 2^S$  es la **función de transición**.

$s_0$  es el **estado inicial**,  $s_0 \in S$ .

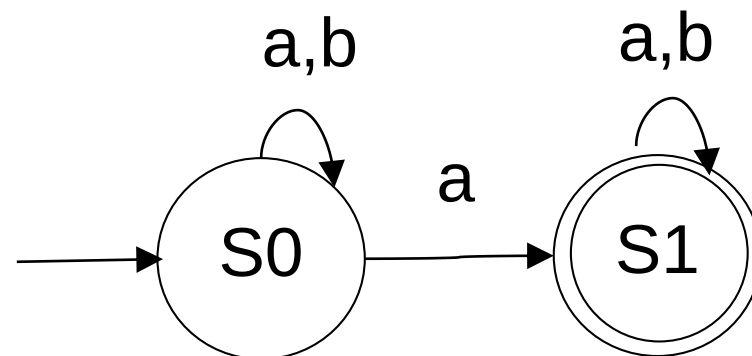
$F$  es el **conjunto de estados finales o aceptadores** ( $F \subseteq S$ ).

## Ejemplo

$$M = (S, \Sigma, \delta, s_0, F),$$

$$\begin{aligned} S &= \{s_0, s_1\}, \\ \Sigma &= \{a, b\}, \\ F &= \{s_1\} \end{aligned}$$

| $\delta$ | a              | b         |
|----------|----------------|-----------|
| $s_0$    | $\{s_0, s_1\}$ | $\{s_0\}$ |
| $s_1$    | $\{s_1\}$      | $\{s_1\}$ |



Autómata Finito  
Reconocedor  
No Determinista

$L(M)$  es el lenguajes de cadenas sobre  $\Sigma = \{a, b\}$   
que contienen al menos una a



Un autómata finito reconocedor no determinista con transiciones  $\lambda$  (AFRND-  $\lambda$ ) es una quintupla

$$M = (S, \Sigma, \delta, s_0, F),$$

donde:

$S$  es un conjunto finito de estados,  $S \neq \emptyset$

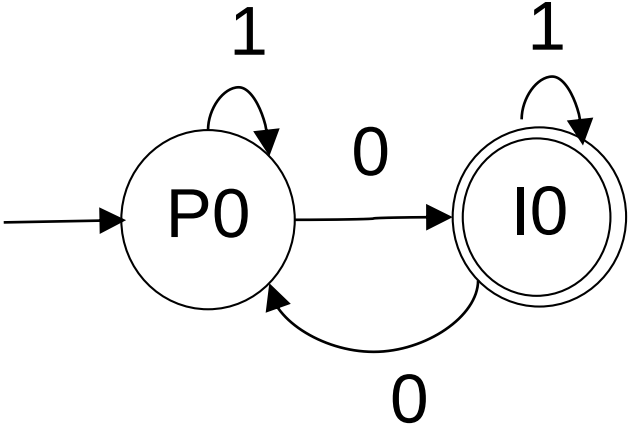
$\Sigma$  es el alfabeto de entrada.

$\delta: S \times (\Sigma \cup \{\lambda\}) \rightarrow 2^S$  es la función de transición.

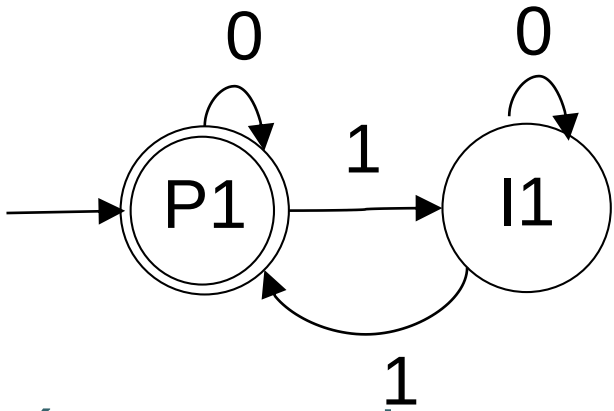
$s_0$  es el estado inicial,  $s_0 \in S$ .

$F$  es el conjunto de estados finales o aceptadores ( $F \subseteq S$ ).

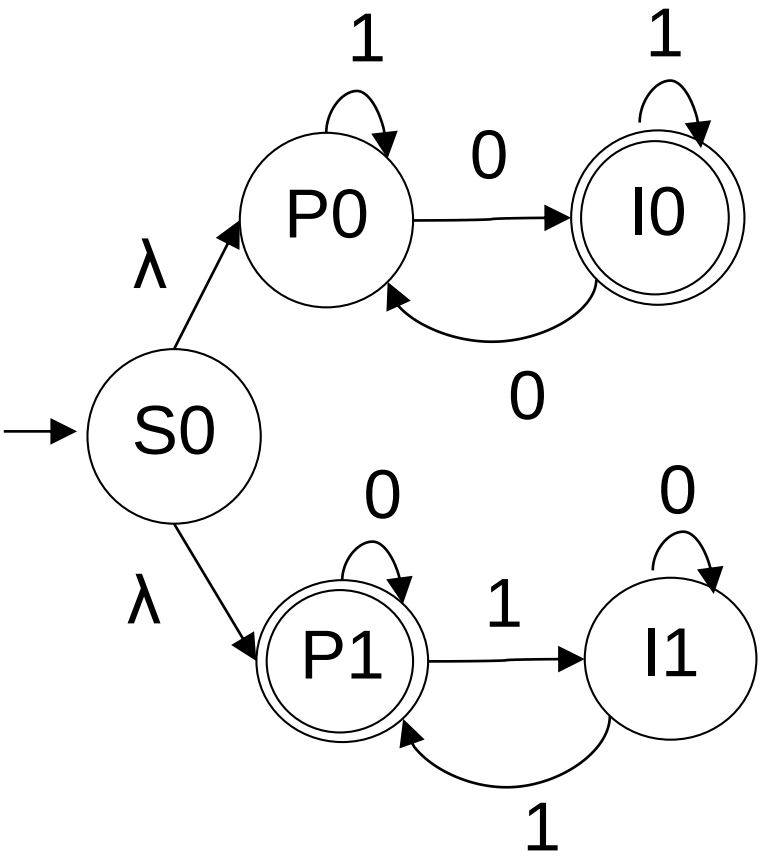
# autómata finito no determinista con transiciones $\lambda$



Número impar de ceros



Número par de unos



Número impar de ceros o  
par de unos

Para AFRND Definimos  $\delta^*: S \times \Sigma^* \rightarrow 2^S$  como sigue:

$$\delta^*(q, \lambda) = \{q\}$$

$$\delta^*(q, wa) = \{r_1, r_2, \dots, r_m\}$$

donde

$$\delta^*(q, w) = \{p_1, p_2, \dots, p_k\} \text{ y}$$

$$\bigcup_{i=1}^k \delta(p_i, a) = \{r_1, r_2, \dots, r_m\}$$

El *lenguaje reconocido* por el AFRND

$M = (S, \Sigma, \delta, s_0, F)$ , se denota  $L(M)$  y se define como:

$$L(M) = \{w \mid \delta^*(s_0, w) \cap F \neq \emptyset\}$$

Es decir, el lenguaje de  $M$  es el conjunto de cadenas  $w$  tales que comenzando del estado inicial de  $M$  nos llevan a algún conjunto de estados que contenga al menos un estado final.

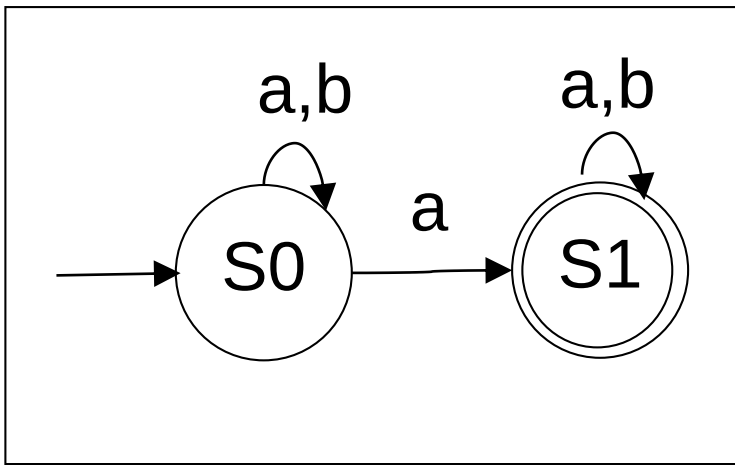
A partir de un AFRND

$N = (S, \Sigma, \delta, s_0, F)$  obtenemos un AFR determinista  $D = (S_d, \Sigma, \delta_d, \{s_0\}, F_d)$

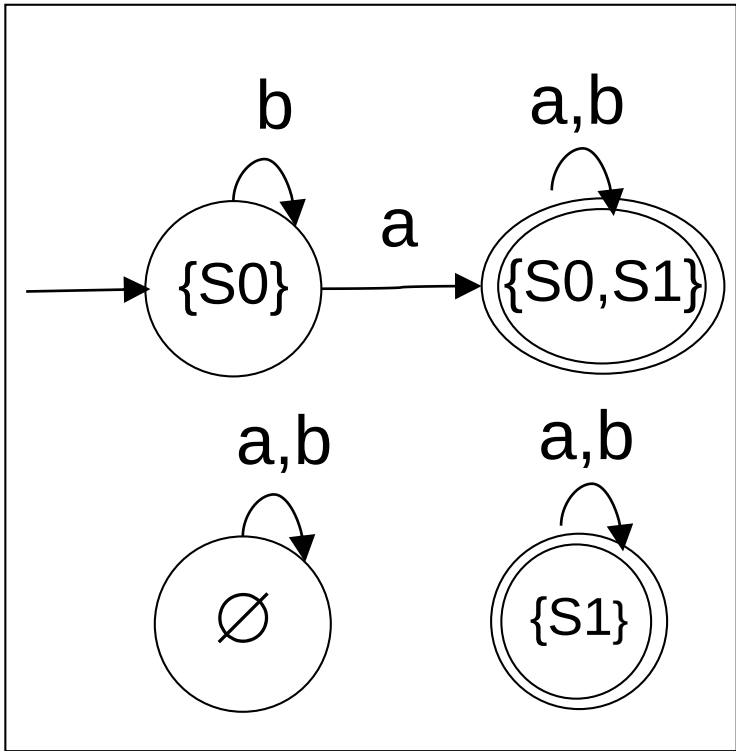
- $S_d = 2^S$
- $F_d$  es el conjunto de subconjuntos  $Q \in S_d$  tal que  $Q \cap F \neq \emptyset$
- Para cada conjunto  $Q \subseteq S$  y para cada símbolo  $a \in \Sigma$

$$\delta_d(Q, a) = \bigcup_{p \in Q} \delta(p, a)$$

$\delta_d$  es una función de transición determinista



AFRND



AFRD

cadenas sobre  $\Sigma = \{a,b\}$  que contienen al menos una  $a$ .

Ejemplo

$N = (S, \Sigma, \delta, s_0, F)$ ,

$S = \{s_0, s_1\},$   
 $\Sigma = \{a, b\},$   
 $F = \{s_1\}$

| $\delta$ | a              | b     |
|----------|----------------|-------|
| $s_0$    | $\{s_0, s_1\}$ | $s_0$ |
| $s_1$    | $s_1$          | $s_1$ |

AFRND

$D = (S_d, \Sigma, \delta_d, \{s_0\}, F_d)$

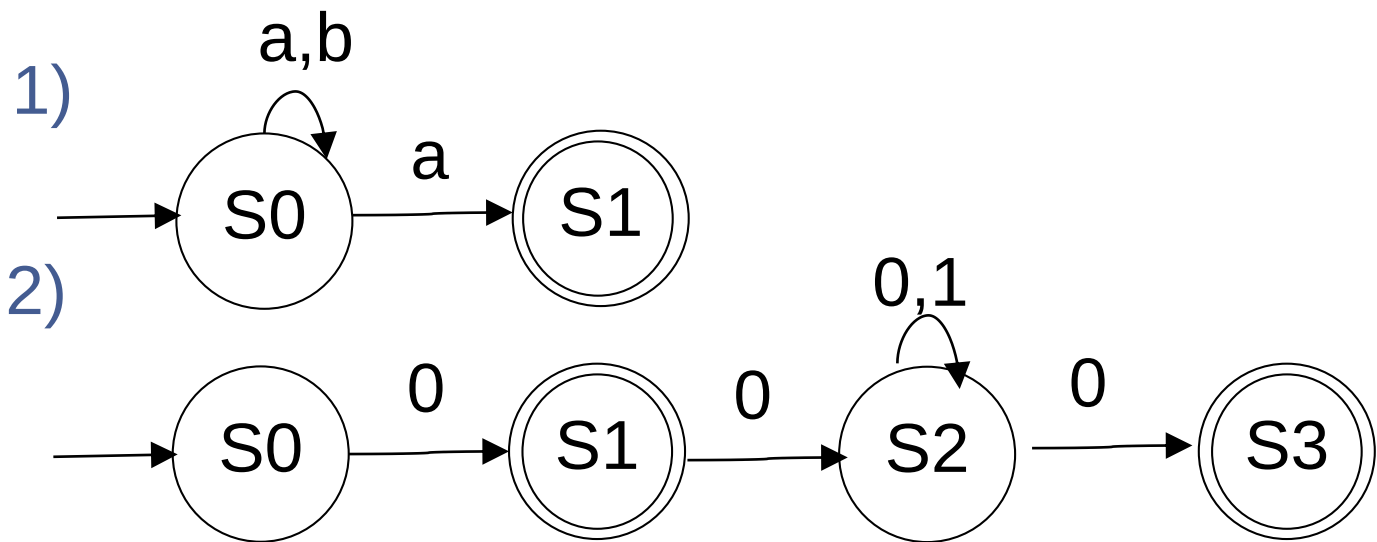
$S_d = \{\emptyset, \{s_0\}, \{s_1\}, \{s_0, s_1\}\},$   
 $\Sigma = \{a, b\},$   
 $F_d = \{\{s_1\}, \{s_0, s_1\}\}$

| $\delta_d$     | a              | b              |
|----------------|----------------|----------------|
| $\emptyset$    | $\emptyset$    | $\emptyset$    |
| $\{s_0\}$      | $\{s_0, s_1\}$ | $\{s_0\}$      |
| $\{s_1\}$      | $\{s_1\}$      | $\{s_1\}$      |
| $\{s_0, s_1\}$ | $\{s_0, s_1\}$ | $\{s_0, s_1\}$ |

AFRD

## Ejercicios

Obtener el AFRD equivalente a los dados a continuación



En lo que sigue usaremos AF para referirnos a autómatas finitos reconocedores (independientemente de si los mismos son deterministas o no deterministas)



La minimización es un proceso que nos permite encontrar, para un dado autómata finito  $M$ , un autómata finito  $M'$  con las siguientes propiedades:

Si  $M$  y  $M'$  comienzan por sus estados iniciales, producirán las mismas salidas para las mismas entradas.

De ser posible  $M'$  tendrá menos estados que  $M$ .

Si esto no es posible, entonces  $M$  ya es un autómata mínimo.

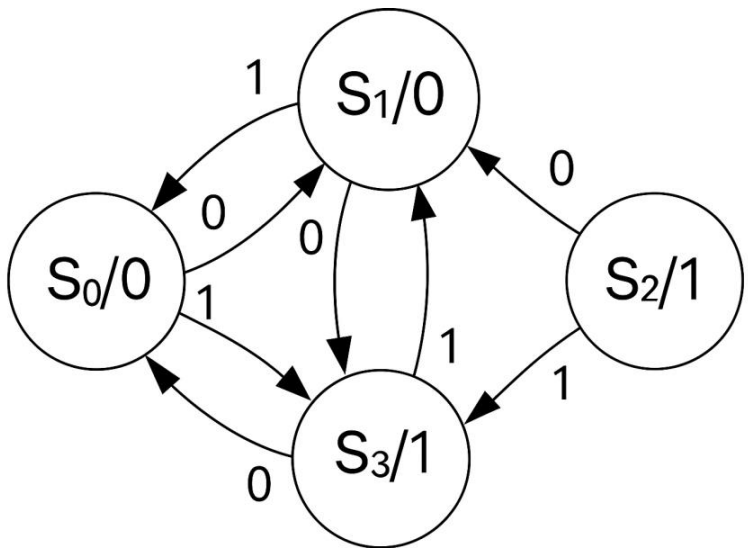
Mostraremos el procedimiento de minimización para autómatas traductores de Moore. El procedimiento para autómatas reconocedores se puede pensar como un caso especial donde se asume que los estados aceptadores emiten la salida 1 y los estados de rechazo emiten la salida 0.

Los estados inalcanzables son aquellos que no pueden alcanzarse desde el estado inicial, independientemente de los símbolos de entrada.

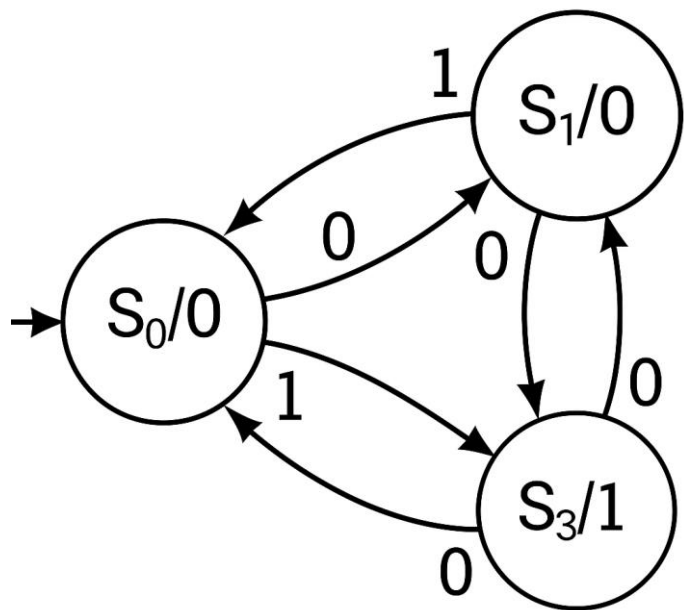
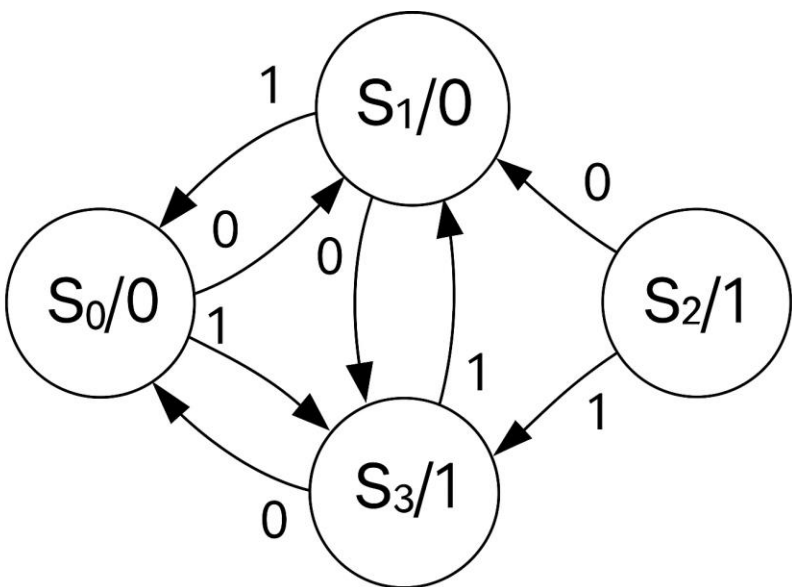
Los estados inalcanzables pueden ser removidos sin alterar las salidas producidas por el autómata.

Tabla para la función de transición:

|    | 0  | 1  | salida |
|----|----|----|--------|
| S0 | S1 | S3 | 0      |
| S1 | S3 | S0 | 0      |
| S2 | S1 | S3 | 1      |
| S3 | S0 | S1 | 1      |



¿Existe algún estado inalcanzable?



Dos estados  $s_i$  y  $s_j$  de  $M$  son equivalentes si para toda  $\alpha \in \Sigma^*$ ,

$$f_O^*(s_i, \alpha) = f_O^*(s_j, \alpha),$$

donde  $\Sigma^*$  denota el conjunto de cadenas de longitud finita sobre el alfabeto de entrada.

De esta manera, estados equivalentes producen salidas equivalentes para una cadena de entrada dada.

Los estados de  $M$  pueden ser particionados en clases de equivalencia tal que:

Todos los estados de la misma clase tienen la misma salida.

La función de transición es tal que para cada símbolo de entrada, todos los estados de la misma clase proceden a estados que están todos en la misma clase.

El problema de minimizar autómatas se reduce al problema de encontrar dichas clases de equivalencia.

El problema se resuelve de manera iterativa, identificando lo que se conoce como clases de estados  $k$ -equivalentes.

Dos estados  $s_i$  y  $s_j$  de  $M$  son  **$k$ -equivalentes** si para todo  $\alpha \in \Sigma^*$ , tal que  $|\alpha| \leq k$ ,  $f_O^*(s_i, \alpha) = f_O^*(s_j, \alpha)$ .

TABLE 8.5

| Present State | Next State    |   | Output |
|---------------|---------------|---|--------|
|               | Present Input |   |        |
|               | 0             | 1 |        |
| 0             | 2             | 3 | 0      |
| 1             | 3             | 2 | 1      |
| 2             | 0             | 4 | 0      |
| 3             | 1             | 5 | 1      |
| 4             | 6             | 5 | 1      |
| 5             | 2             | 0 | 0      |
| 6             | 4             | 0 | 1      |

0-equivalentes: {0, 2, 5} y {1, 3, 4, 6}.

1-equivalentes: {0, 2},{5},{1, 3, 4, 6}.

2-equivalentes: {0, 2},{5},{1, 6},{3, 4}.

3-equivalentes: {0, 2},{5},{1, 6},{3, 4}.



**Entrada:** un AFD completo  $M=(S, \Sigma, \delta, s_0, f_0)$

**Salida:** un AF  $M'$  equivalente a  $M$  minimizado

1)  $T \leftarrow S - \{\text{estados inalcanzables en } S\}$

2) Particionar  $T$  en clases formadas por estados 0-equivalentes

3)  $k \leftarrow 0$

4) **Repetir**

Determinar clases  $(k+1)$ -equivalentes como refinamiento de las clases  $k$ -equivalentes

$s_i, s_j$  son  $(k+1)$ -equivalentes sssi

$s_i, s_j$  son  $k$ -equivalentes y  $\delta(s_i, a), \delta(s_j, a)$  son  $k$ -equivalentes  $\forall a \in \Sigma$

$k \leftarrow k+1$

**Hasta**  $\text{Clases}((k+1)\text{-equivalentes}) = \text{Clases}(k\text{-equivalentes})$

5) Definir  $M'$  usando las clases  $k$ -equivalentes halladas

La máquina resultante de reducir  $M$  del ejercicio anterior tendrá los siguientes estados

$$A = \{0, 2\}; B = \{5\}; C = \{1, 6\}; D = \{3, 4\}$$

Tendrá 4 estados en lugar de 7 y la siguiente tabla de transición:

TABLE 8.6

| Present State | Next State    |          | Output |
|---------------|---------------|----------|--------|
|               | Present Input |          |        |
|               | 0             | 1        |        |
| <i>A</i>      | <i>A</i>      | <i>D</i> | 0      |
| <i>B</i>      | <i>A</i>      | <i>A</i> | 0      |
| <i>C</i>      | <i>D</i>      | <i>A</i> | 1      |
| <i>D</i>      | <i>C</i>      | <i>B</i> | 1      |

TABLE 8.7

| Present State | Next State    |   | Output |
|---------------|---------------|---|--------|
|               | Present Input |   |        |
|               | 0             | 1 |        |
| 0             | 3             | 1 | 1      |
| 1             | 4             | 1 | 0      |
| 2             | 3             | 0 | 1      |
| 3             | 2             | 3 | 0      |
| 4             | 1             | 0 | 1      |

0-equivalentes:  
{0, 2, 4}, {1, 3}

1-equivalentes:  
{0}, {2, 4}, {1, 3}

2-equivalentes:  
{0}, {2, 4}, {1, 3}

A = {0},  
B = {2, 4},  
C = {1, 3}

TABLE 8.8

| Present State | Next State    |          | Output |
|---------------|---------------|----------|--------|
|               | Present Input |          |        |
|               | 0             | 1        |        |
| <i>A</i>      | <i>C</i>      | <i>C</i> | 1      |
| <i>B</i>      | <i>C</i>      | <i>A</i> | 1      |
| <i>C</i>      | <i>B</i>      | <i>C</i> | 0      |

Las **expresiones regulares** (ER) sobre un alfabeto  $\Sigma$  son cadenas sobre el alfabeto  $A = \Sigma \cup \{ (, ), \cdot, +, *, \lambda, \emptyset \}$  definidas recursivamente como sigue:

1.  $\lambda$  es una ER.
2.  $\emptyset$  es una ER.
3. Todo símbolo  $a \in \Sigma$  es una ER.
4. Si  $\alpha$  y  $\beta$  son ER, entonces  $(\alpha)$ ,  $\alpha \cdot \beta$ ,  $\alpha + \beta$  y  $\alpha^*$  también son ER.

Construimos la función  $L$  tal que si  $\alpha$  es una ER entonces  $L(\alpha)$  es el lenguaje representado por  $\alpha$ :

1.  $L(\lambda) = \{\lambda\}$ .
2.  $L(\emptyset) = \emptyset$ .
3. Para cada símbolo  $a \in \Sigma$ ,  $L(a) = \{a\}$ .
4. Si  $\alpha$  y  $\beta$  son ER, entonces
  - $L(\alpha \cdot \beta) = L(\alpha) \cdot L(\beta)$ ,
  - $L(\alpha + \beta) = L(\alpha) \cup L(\beta)$ ,
  - $L(\alpha^*) = L(\alpha)^*$ .

Algunos ejemplos de ER sobre el alfabeto  $\Sigma=\{a,b\}$ :

$\lambda$ ,  $\emptyset$ ,  $a^+ \lambda$ ,  $a^* \cdot (a+b)^* \cdot b$ ,  $b \cdot a^*$

## Observaciones:

1. El símbolo “ $\cdot$ ” (concatenación) podrá ser obviado cuando ello no provoque ambigüedad. Por ejemplo la ER  $b \cdot a^*$  se escribirá como  $ba^*$ .
2. Notar que el mismo lenguaje regular puede ser notado mediante diferentes expresiones regulares. Por ejemplo  $L((0+1)^*) = L(((0+1^*)^* + (01)^*)^*)$

## Teorema de Kleene

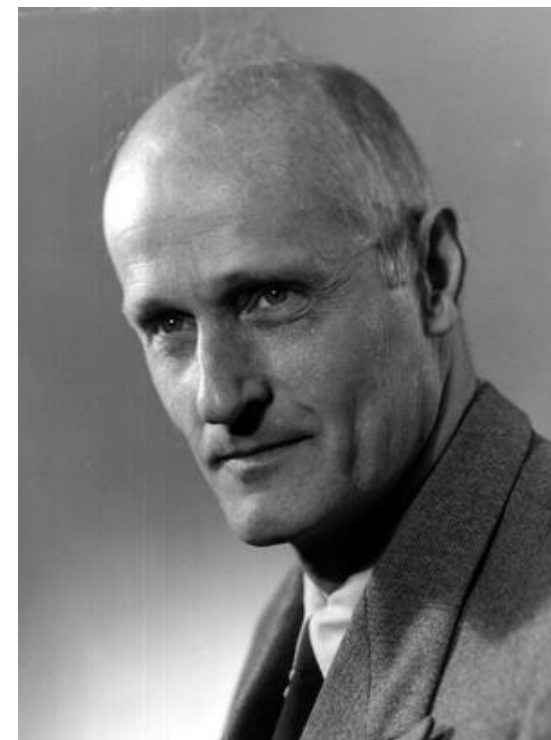
(detalles en Hopcroft et al., sección 3.2):

Un lenguaje  $L$  puede ser expresado por una ER si y solo si es reconocido por un AF.

## Prueba

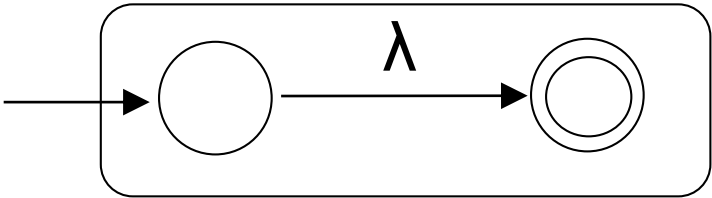
Parte 1: Dada una ER, construimos un AF.

Parte 2: Dado un AF, construimos una ER.



Stephen Kleene  
1909-1994

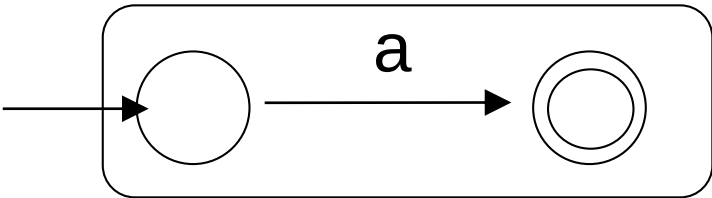
1.  $\lambda$



2.  $\emptyset$

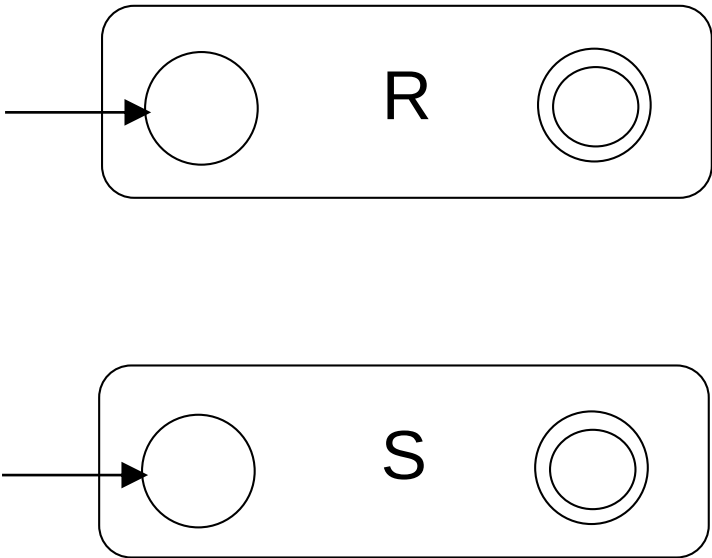


3.  $a \in \Sigma$

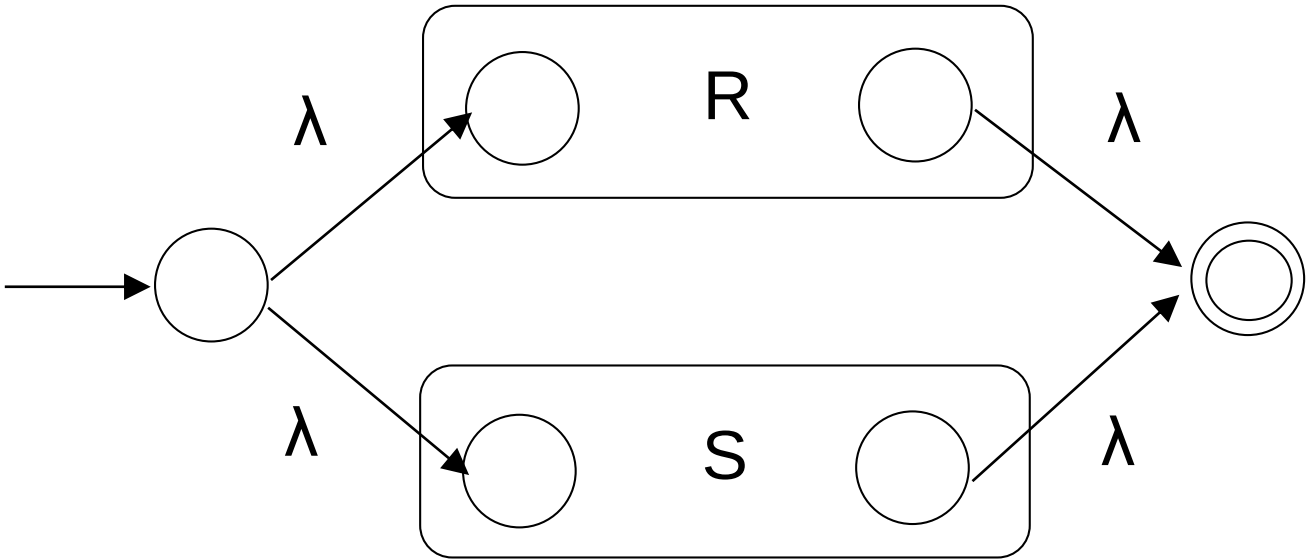




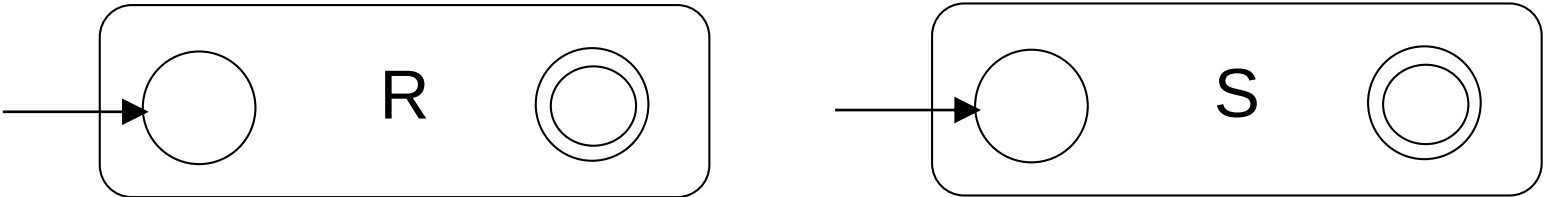
4.1.  $\alpha + \beta$



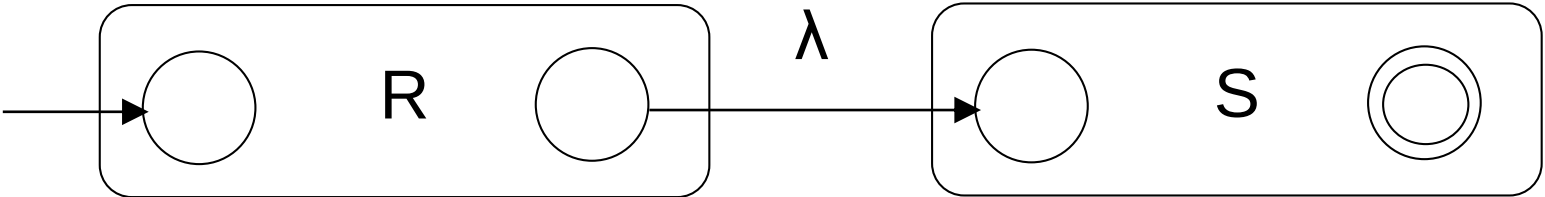
4.1.  $\alpha + \beta$



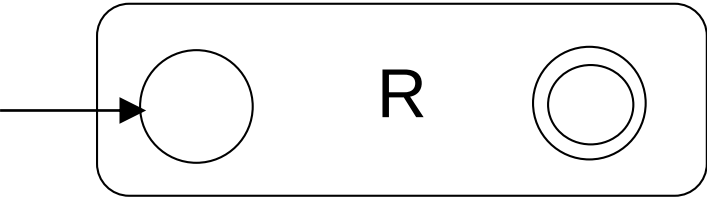
4.2.  $\alpha \cdot \beta$



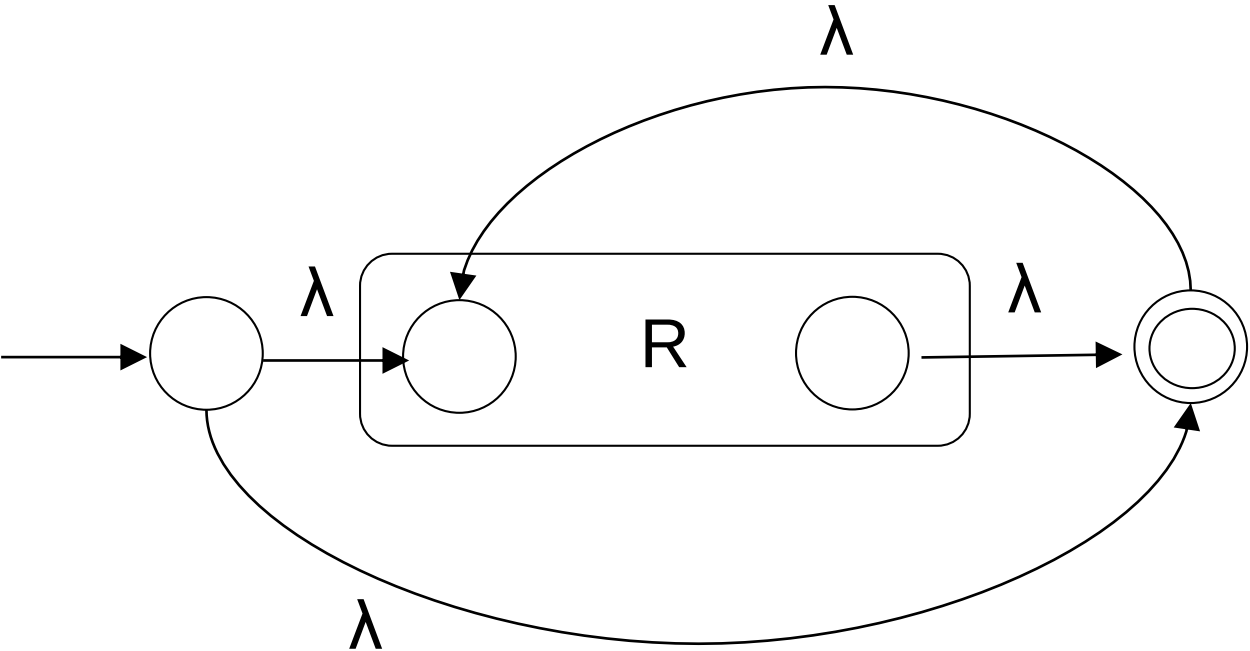
4.2.  $\alpha \cdot \beta$



4.3.  $a^*$

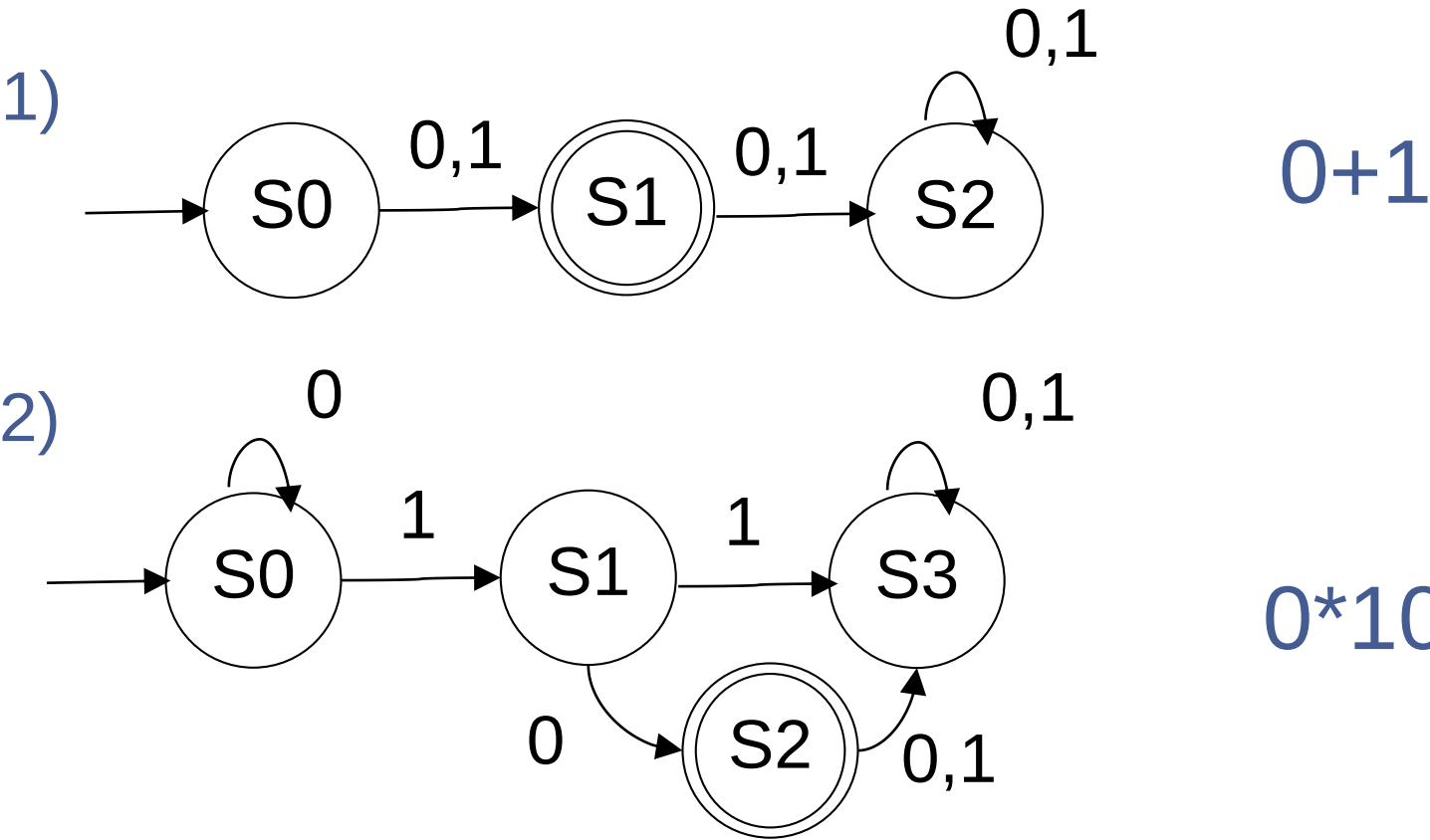


4.3.  $\alpha^*$



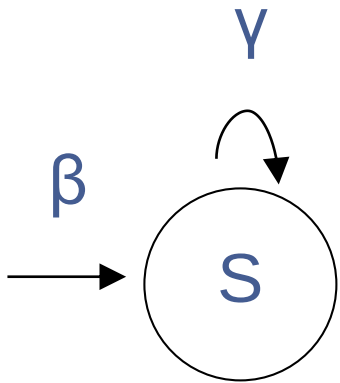
**Ejemplo:** AF  $\rightarrow$  ER (informal)

Escribir la expresión regular para los siguientes AF



# Lema

Si  $\omega$ ,  $\beta$  y  $\gamma$  son ER sobre un alfabeto  $\Sigma$ ,  $\gamma \neq \lambda$ , entonces la ecuación  $\omega = \beta + \omega\gamma$  tiene una única solución y está dada por  $\omega = \beta\gamma^*$



asociar cada estado  
con una ER

$$\omega_S = \beta + \omega_S\gamma \quad \longrightarrow \quad \omega_S = \beta\gamma^*$$



## Algoritmo

**Entrada:** un AF  $M = (S, \Sigma, \delta, s_0, F)$

**Salida:** una ER que denota el LR reconocido por M.

## Paso 1:

Plantear una ecuación por cada estado, como unión de  $n$  términos

$$\omega s_i = \omega s_{j_1} \cdot a_1 + \omega s_{j_2} \cdot a_2 \dots$$

Para  $\delta(s_j, a) = s_i$ , uno de los términos de la ecuación  $\omega s_i$  será  $\omega s_j \cdot a$ .

En la ecuación para el estado inicial se agrega el término  $\lambda$ .

## Paso 2:

despejar ecuaciones según lema  $\omega = \beta + \omega\gamma$   
 entonces  $\omega = \beta\gamma^*$

## Paso 3:

La ER es la unión de las soluciones para todos los estados aceptadores del AF.

## Ejemplo

$M = (S, \Sigma, \delta, s_0, F)$

$S = \{s_0, s_1, s_2\}, \Sigma = \{a, b\}, F = \{s_1\}$

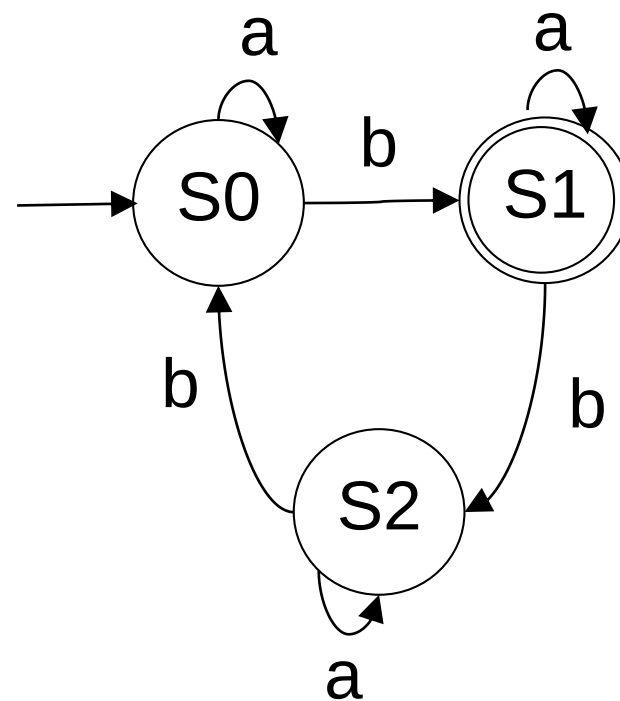
| $\delta$ | a     | b     |
|----------|-------|-------|
| $s_0$    | $s_0$ | $s_1$ |
| $s_1$    | $s_1$ | $s_2$ |
| $s_2$    | $s_2$ | $s_0$ |

**Paso 1:** Planteamos ecuaciones

$$\omega_{s_0} = \omega_{s_0}a + \omega_{s_2}b + \lambda$$

$$\omega_{s_1} = \omega_{s_1}a + \omega_{s_0}b$$

$$\omega_{s_2} = \omega_{s_2}a + \omega_{s_1}b$$



**Paso 2:**  $\omega = \beta + \omega\gamma$  entonces  $\omega = \beta\gamma^*$

$$\omega_{S0} = \omega_{S0}a + \omega_{S2}b + \lambda \longrightarrow \omega_{S0} = (\omega_{S2}b + \lambda)a^*$$

$$\gamma = a \text{ y } \beta = \omega_{S2}b + \lambda$$

$$\omega_{S1} = \omega_{S1}a + \omega_{S0}b \longrightarrow \omega_{S1} = (\omega_{S0}b)a^*$$

$$\gamma = a \text{ y } \beta = \omega_{S0}b$$

$$\omega_{S2} = \omega_{S2}a + \omega_{S1}b \longrightarrow \omega_{S2} = (\omega_{S1}b)a^*$$

$$\gamma = a \text{ y } \beta = \omega_{S1}b$$

$$\omega_{S1} = (\omega_{S0}b)a^*$$

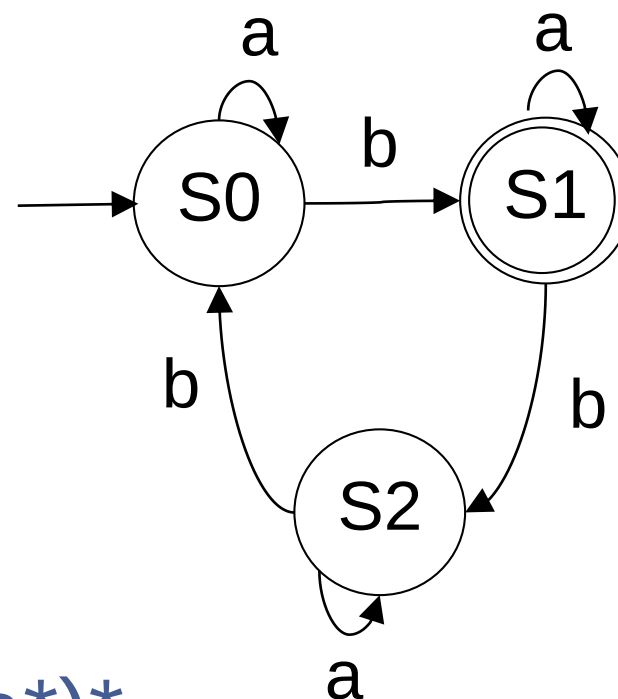
$$\omega_{S1} = ((\omega_{S2}b + \lambda)a^*b)a^*$$

$$\omega_{S1} = (((\omega_{S1}b)a^*b + \lambda)a^*b)a^* = \omega_{S1}ba^*ba^*ba^* + a^*ba^*$$

$$\omega_{S1} = a^*ba^*(ba^*ba^*ba^*)^*$$

**Paso 3:** La ER es la unión de las soluciones para todos los estados aceptadores del AF.

$$\omega_{S_1} = a^*ba^*(ba^*ba^*ba^*)^*$$



$$L(M) = a^*ba^*(ba^*ba^*ba^*)^*$$

**Lema:** La clase de los lenguajes aceptados por AF, es decir la clase de los lenguajes regulares, es cerrada bajo:

unión,  
concatenación,  
estrella de Kleene,  
complemento e  
intersección.

A continuación daremos una prueba para unión. Queda como **ejercicio** demostrar que el lema se verifica para las otras operaciones.

Demostramos que la clase de lenguajes regulares es cerrada bajo unión.

**Definición:** Un lenguaje  $L$  es regular sssi existe un autómata finito  $M$  tal que  $L(M) = L$

**Prueba:**

Sean  $L_1$  y  $L_2$  dos lenguajes regulares. Probaremos que  $L_1 \cup L_2$  también es un lenguaje regular. Por definición de lenguaje regular, es suficiente probar que es posible construir un autómata finito  $M'$  tal que  $L(M') = L_1 \cup L_2$ .

continúa en la próxima diapositiva



Dado que  $L_1$  y  $L_2$  son lenguajes regulares, por definición de lenguaje regular, existen autómatas finitos

$M_1 = (\mathbf{S}^1, \Sigma, \delta^1, s_0^1, \mathbf{F}^1)$  y  $M_2 = (\mathbf{S}^2, \Sigma, \delta^2, s_0^2, \mathbf{F}^2)$  tales que

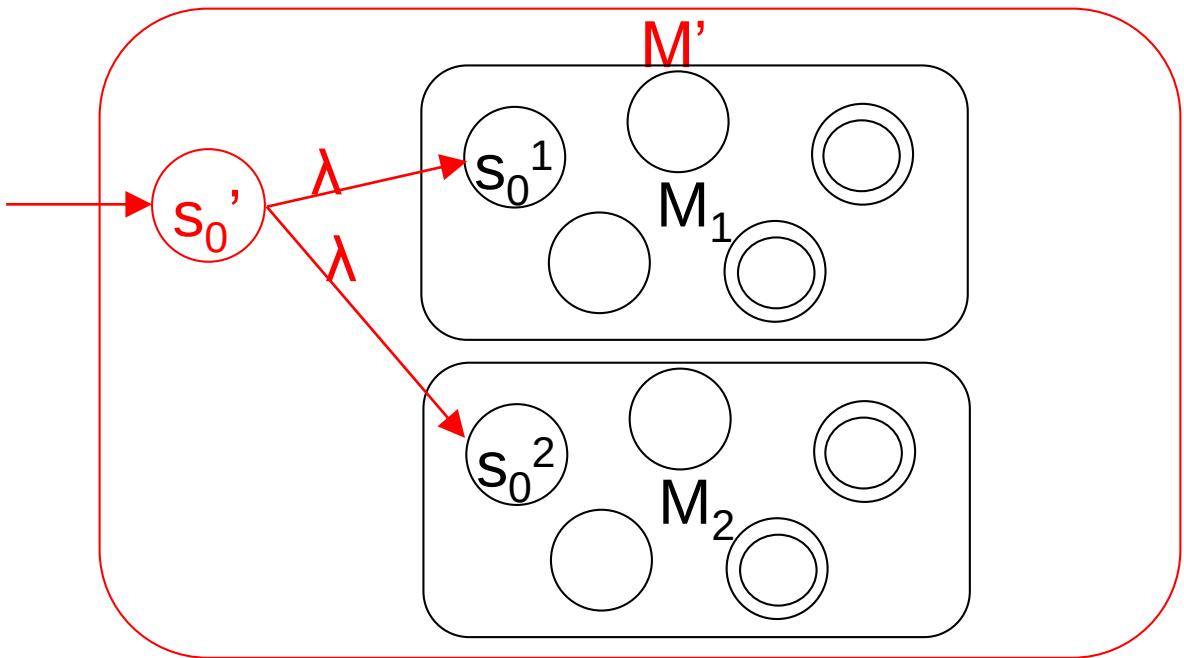
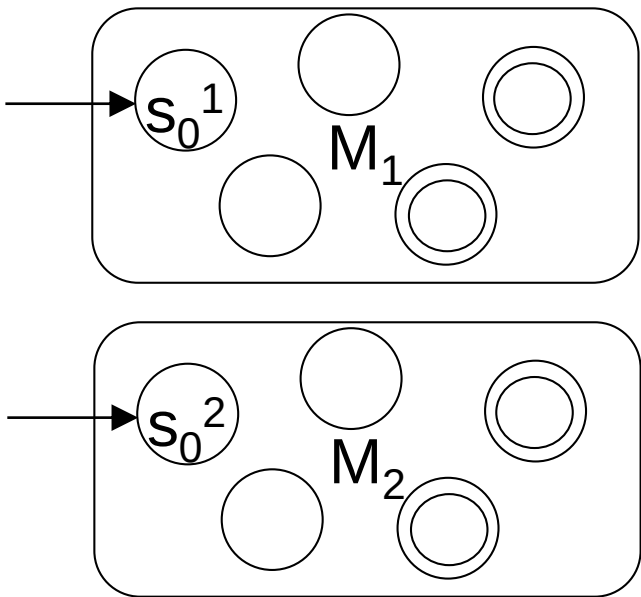
$L(M_1) = L_1$  y  $L(M_2) = L_2$ .

Sin pérdida de generalidad podemos asumir que el alfabeto  $\Sigma$  es común a ambos autómatas y que  $\mathbf{S}^1 \cap \mathbf{S}^2 = \emptyset$ . A partir de los autómatas finitos  $M_1$  y  $M_2$  construimos  $M' = (\mathbf{S}', \Sigma, \delta', s_0', \mathbf{F}')$ , donde:

$\mathbf{S}' = \mathbf{S}^1 \cup \mathbf{S}^2 \cup \{s_0'\}$  donde  $s_0'$  es el nuevo estado inicial,  
 $s_0' \notin \mathbf{S}^1 \cup \mathbf{S}^2$

$\delta' = \delta^1 \cup \delta^2 \cup \{ \delta(s_0', \lambda) = s_0^1 \} \cup \{ \delta(s_0', \lambda) = s_0^2 \}$

$\mathbf{F}' = \mathbf{F}^1 \cup \mathbf{F}^2$



Demostraremos ahora que la construcción es correcta, es decir,  $L(M') = L_1 \cup L_2$ . Dado que  $L_1 = L(M_1)$  y  $L_2 = L(M_2)$ , esto es equivalente a demostrar que  $L(M') = L(M_1) \cup L(M_2)$ . Probaremos entonces que  $w \in L(M') \iff w \in L(M_1) \cup L(M_2)$  para todo  $w \in \Sigma^*$ .

$$\begin{aligned} w \in L(M') &\iff \delta'^*(s_0', w) \in F' \iff \delta^{1*}(s_0^1, w) \in F^1 \text{ o } \delta^{2*}(s_0^2, w) \in F^2 \\ &\iff w \in L(M_1) \text{ o } w \in L(M_2) \iff w \in L(M_1) \cup L(M_2). \end{aligned}$$

Queda demostrado que para todo  $w \in \Sigma^*$  se verifica  

$$w \in L(M') \iff w \in L(M_1) \cup L(M_2).$$

Una gramática formal consiste en un conjunto de reglas destinadas a transformar secuencias de caracteres, junto con un símbolo inicial que marca el punto de inicio para llevar a cabo estas transformaciones. En consecuencia, una gramática formal suele considerarse como un mecanismo generador de lenguajes.

Una **gramática** para un lenguaje puede ser descripta definiendo su proceso generativo.

Una gramática estructura de frase (tipo 0) es una 4-tupla

$G = (V_N, V_T, S, P)$ , donde

- $V_N$  = conjunto de símbolos no terminales.
- $V_T$  = conjunto de símbolos terminales.
- $S$  = símbolo inicial de la gramática.
- $P$  = conjunto finito de producciones de la forma  $\alpha \rightarrow \beta$  donde  $\alpha$  es una cadena sobre  $V_N UV_T$  con al menos un símbolo de  $V_N$  y  $\beta$  es una cadena sobre  $V_N UV_T$ .  
(usamos  $\alpha \rightarrow \lambda$  cuando  $\beta$  es la cadena nula)

Sea  $G=(V_N, V_T, S, P)$  y sean  $w_1$  y  $w_2$  cadenas sobre  $V_N \cup V_T$ .

Decimos que  $w_1$  **deriva directamente**  $w_2$ , y lo notamos  $w_1 \Rightarrow w_2$  si

- $\alpha \rightarrow \beta$  es una producción en  $P$ ,
- $w_1$  contiene una instancia de  $\alpha$ ,
- $w_2$  es obtenida a partir de  $w_1$  reemplazando esa instancia de  $\alpha$  con  $\beta$ .

Si  $w_1, w_2, \dots, w_n$  son cadenas sobre  $V_N \cup V_T$  y  $w_1 \Rightarrow w_2$ ,  
 $w_2 \Rightarrow w_3, \dots, w_{n-1} \Rightarrow w_n$ , entonces  $w_1$  **genera (deriva)**  $w_n$ .

Se escribe  $w_1 \overset{+}{\Rightarrow} w_n$  si es en 1 o más pasos.

Se escribe  $w_1 \overset{*}{\Rightarrow} w_n$  si es en 0 o más pasos.

Dada una gramática  $G$ , el lenguaje  $L$  generado por  $G$ , denotado  $L(G)$ , es el conjunto  $L = \{w \in V_T^* \mid S \xRightarrow{+} w\}$ .

En otras palabras,  $L$  es el conjunto de todas las cadenas de terminales generadas a partir de  $S$ .

**Observación:** Una vez que una cadena  $w$  de terminales ha sido obtenida, ninguna producción puede ser aplicada a  $w$ , y  $w$  no puede generar otras palabras.



Sea  $L = \{a^{2n} \mid n \geq 0\}$ .

Una gramática para  $L$  es  $G=(V_N, V_T, S, P)$  donde  
 $V_N = \{S\}$ ,  $V_T = \{a\}$ , y  $P$  consiste de las siguientes producciones:

$$S \rightarrow \lambda \mid aaS$$

Podemos generar aaaaaa como sigue:

$$S \Rightarrow aaS$$

$$\Rightarrow aaaaS$$

$$\Rightarrow aaaaaaS$$

$$\Rightarrow aaaaaa$$

Sea  $L = \{a^m b^n \mid m, n \geq 0\}$ .

Una gramática para  $L$  es  $G=(V_N, V_T, S, P)$  donde  $V_N = \{S, B\}$ ,  $V_T = \{a, b\}$ , y  $P$  consiste de las siguientes producciones:

$$S \rightarrow \lambda \mid aS \mid B$$

$$B \rightarrow bB \mid b$$

Podemos generar  $abb$  como sigue:

$$S \Rightarrow aS$$

$$\Rightarrow aB$$

$$\Rightarrow abB$$

$$\Rightarrow abb$$

Sea  $L = \{0^n 1^n \mid n \geq 1\}$ .

Una gramática para  $L$  es  $G=(V_N, V_T, S, P)$  donde  
 $V_N = \{S\}$ ,  $V_T = \{0, 1\}$ , y  $P$  consiste de las siguientes producciones:

$$S \rightarrow 0S1 \mid 01$$

Podemos generar  $0^3 1^3$  como sigue:

$$\begin{aligned} S &\Rightarrow 0S1 \\ &\Rightarrow 00S11 \\ &\Rightarrow 000111 \end{aligned}$$

- **Estructura de frase (tipo 0).** Sin restricciones.
- **Sensible al contexto (tipo 1).** Para cada producción  $\alpha \rightarrow \beta$ :  $|\alpha| \leq |\beta|$  (excepto  $S \rightarrow \lambda$ ).
- **Libre de contexto (tipo 2).** Para cada producción  $\alpha \rightarrow \beta$ :  $\alpha \in V_N$  y  $|\alpha| \leq |\beta|$  (excepto  $S \rightarrow \lambda$ ).
- **Regular (tipo 3).** Para cada producción  $\alpha \rightarrow \beta$ :  $\alpha \in V_N$  y  $\beta$  es de la forma  $t$  o  $tW$ , donde  $t \in V_T$  y  $W \in V_N$  (excepto  $S \rightarrow \lambda$ ).

# Ejercicios

Determinar qué lenguaje genera cada una de las siguientes gramáticas  $G=(V_N, V_T, S, P)$  . Determinar su tipo.

1)

$$V_N = \{S\},$$

$$V_T = \emptyset$$

$$P: S \rightarrow \lambda$$

2)

$$V_N = \{S\}$$

$$V_T = \emptyset$$

$$P = \emptyset$$

3)

$$V_N = \{S\}$$

$$V_T = \{a\}$$

$$P : S \rightarrow a \mid aS$$

4)

$$V_N = \{S, A\},$$

$$V_T = \{a\}$$

$$P : S \rightarrow \lambda \mid aA$$

$$A \rightarrow aS$$

5)

$$V_N = \{S\},$$

$$V_T = \{a, b\}$$

$$P : S \rightarrow \lambda \mid SS \mid aSb \mid bSa$$

6)

$$V_N = \{S\},$$

$$V_T = \{a, b\}$$

$$P : S \rightarrow aS$$

$$aS \rightarrow bbb$$

7)

$$V_N = \{S\},$$

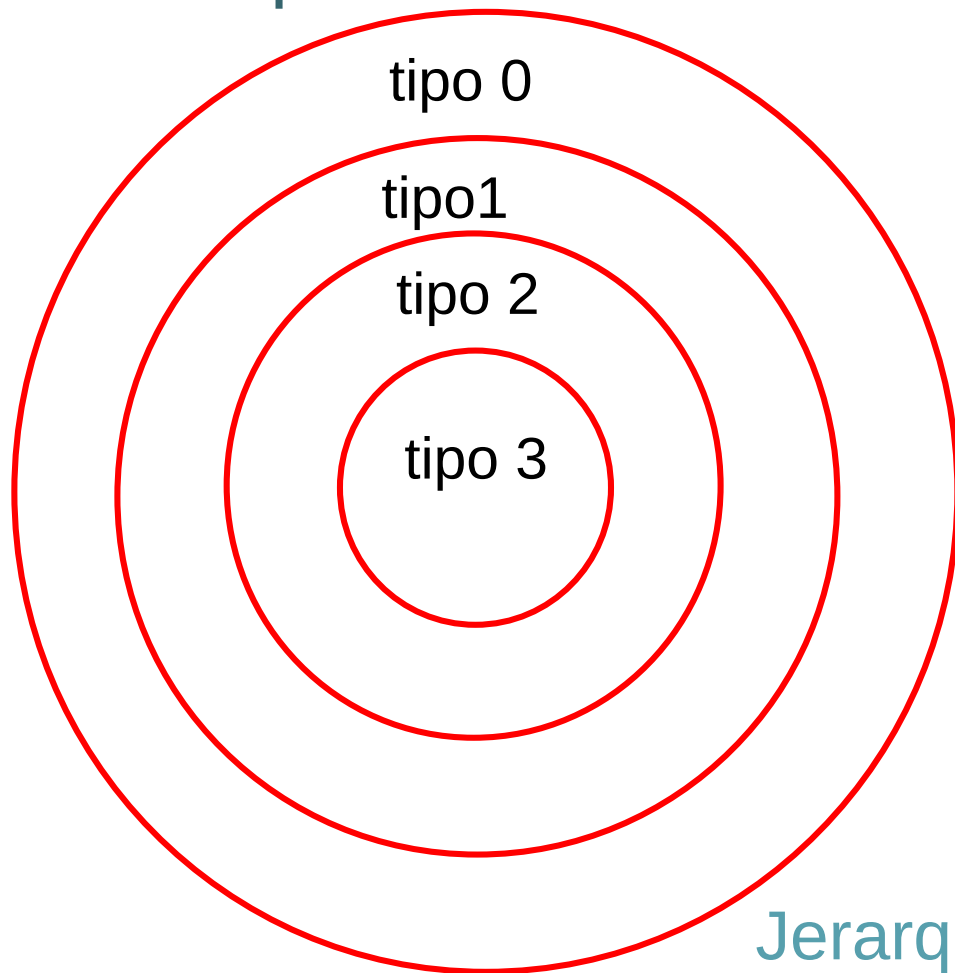
$$V_T = \{a, b\}$$

$$P : S \rightarrow aS$$

$$aS \rightarrow b$$

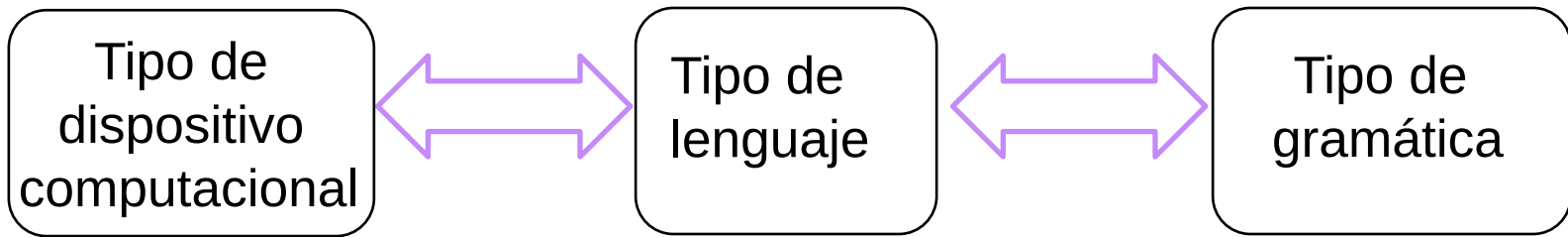


Un lenguaje se dice de tipo N si puede ser generado por una gramática de tipo N



Noam Chomsky  
(1928-)

Jerarquía de Chomsky



El dispositivo computacional más general se llama **máquina de Turing** (se verá en Teoría de Computabilidad) y se corresponde con los lenguajes de tipo 0.

Los lenguajes reconocidos por máquinas de Turing son los lenguajes de tipo 0.

El dispositivo computacional más básico es el **autómata finito** y se corresponde con los lenguajes de tipo 3.

Existen dispositivos computacionales con capacidad intermedia entre aquellos reconocidos por autómatas finitos y máquinas de Turing (se verán en Teoría de Computabilidad):

- **Autómatas a Pila** para lenguajes de tipo 2
- **Máquinas de Turing acotadas linealmente** para lenguajes de tipo 1.

Las gramáticas regulares sirven para generar:

- Cualquier lenguaje finito (aunque puede volverse una tarea tediosa).
- Lenguajes infinitos que presentan ciertas regularidades que pueden ser expresada de manera sencilla utilizando las llamadas expresiones regulares (como veremos más adelante).

Ejemplos de lenguajes que pueden ser generados por gramáticas regulares:

- $L = \emptyset$ . El lenguaje vacío.
- $L = \Sigma^*$  para cualquier  $\Sigma$ . El lenguaje universal.
- $L = \{w \mid w \in \{a,b\}^* \text{ donde } w \text{ tiene } m \text{ a's seguidas de } n \text{ b's para } m, n \geq 0\}$ .
- $L = \{w \mid w \in \{0,1\}^* \text{ donde } w \text{ tiene exactamente tres 0's y tres 1's}\}$ .

Las gramáticas libres de contexto tienen ciertas características destacadas:

Son sencillas, reemplazan un símbolo por vez.

Pueden ser utilizadas para describir la sintaxis de los lenguajes de programación.

La derivación de una gramática libre de contexto puede ser vista como un árbol de derivación.

Ejemplo de lenguajes que pueden ser generados por gramáticas libres de contexto:

- Todos los que pueden ser generados por gramáticas regulares
- $L = \{a^n b^n \mid n \geq 0\}$
- $L = \{w \mid w \in \{(\,,)\}^* \text{ donde los paréntesis están balanceados}\}$
- $L = \{w \mid w \in \{a,b\}^* \text{ y } w = w^r\}$ .

Observación: Si  $w = w_1 w_2 \dots w_n$ ,  
 $w_i \in \{a,b\}$ , entonces  $w^r = w_n w_{n-1} \dots w_1$

## Ejemplo

Sea  $L = \{ab, aabb, aaabbb, \dots\}$  el lenguaje de cadenas que contienen  $n$  a's seguidas de  $n$  b's con  $n \geq 1$ .

Este lenguaje también puede denotarse como  $L = \{a^n b^n \mid n \geq 1\}$

La gramática  $G$  es la siguiente  $G = (V_N, V_T, S, P)$

$V_N = \{S\}, V_T = \{a, b\}$

$P = \{S \rightarrow ab \mid aSb\}$

Probar por inducción que  $L(G) = L$



$$L(G) \subseteq L$$

Toda secuencia de símbolos que podemos derivar a partir de  $S$  consiste en cadenas de la forma  $a^k b^k$ . La prueba se realiza por inducción sobre el número de pasos en la derivación.

Caso base:  $n=1$   $S \Rightarrow ab$ , es de la forma  $a^1 b^1$

Hipótesis inductiva: Supongamos que derivaciones con  $n$  pasos producen cadenas de la forma  $a^k b^k$ .

Paso inductivo: Consideremos una derivación con  $n+1$  pasos

$$S \Rightarrow aSb \xRightarrow{n} axb$$

Por la hipótesis inductiva  $x$  es de la forma  $a^k b^k$  y por lo tanto  $axb = a^{k+1} b^{k+1}$ .

$L \subseteq L(G)$

Toda cadena de la forma  $a^k b^k$  puede derivarse a partir de  $S$ .  
La prueba se realiza por inducción sobre la longitud de la cadena.

Caso base:  $|\omega| = 2, S \Rightarrow ab$

Hipótesis inductiva: Asumamos que  $a^n b^n$  de longitud  $2n$  puede ser derivada a partir de  $S$

Paso inductivo: Consideremos una cadena  $\omega$  de longitud  $2(n+1)$ . Entonces  $\omega = a^{n+1} b^{n+1} = a a^n b^n b$ . Dado que  $|a^n b^n| = 2n$ , puede derivarse a partir de  $S$  por la hipótesis inductiva. Por lo tanto  $S \Rightarrow aSb \stackrel{+}{\Rightarrow} a a^n b^n b = \omega$ . Concluimos que  $\omega$  puede derivarse a partir de  $S$ .

## Ejemplo

Sea  $L = \{\lambda, (), ()(), (()), (())(), \dots\}$  el lenguaje de los paréntesis bien balanceados.

La gramática  $G$  es la siguiente  $G = (V_N, V_T, S, P)$

$$V_N = \{S\}, V_T = \{ (, ) \}$$

$$P = \{ S \rightarrow (S)S \mid \lambda \}$$

Probar que  $L(G) = L$

$$L(G) \subseteq L$$

Toda secuencia de símbolos que podemos derivar a partir de  $S$  consiste en cadenas de paréntesis bien balanceados. La prueba se realiza por inducción sobre el número de pasos en la derivación.

Caso base:  $n=1$   $S \Rightarrow \lambda$ , es balanceado

Hipótesis inductiva: Supongamos que derivaciones con  $n$  o menos pasos producen cadenas balanceadas.

Paso inductivo: Consideremos una derivación con  $n+1$  pasos

$$S \Rightarrow (S)S \xRightarrow{n} (x)y$$

Las cadenas  $x$  e  $y$  fueron derivadas en  $n$  o menos pasos. Por la hipótesis inductiva  $x$  e  $y$  son balanceadas y por lo tanto  $(x)y$  es balanceada.

$$L \subseteq L(G)$$

Toda cadena balanceada puede derivarse a partir de  $S$ . La prueba se realiza por inducción sobre la longitud de la cadena.

Caso base:  $|\omega| = 0, S \Rightarrow \lambda$

Hipótesis inductiva: Asumamos que toda cadena balanceada de longitud menor o igual que  $2n$  puede ser derivada a partir de  $S$

Paso inductivo: Consideremos una cadena balanceada  $\omega$  de longitud  $2(n+1)$  y sea  $(x)$  el prefijo más corto de  $\omega$  que está bien balanceado. Entonces  $\omega = (x)y$ , donde  $x$  e  $y$  están balanceados. Dado que  $|x|, |y| \leq 2n$ , pueden derivarse a partir de  $S$  por la hipótesis inductiva. Por lo tanto

$$S \Rightarrow (S)S \overset{+}{\Rightarrow} (x)S \overset{+}{\Rightarrow} (x)y = \omega.$$

Concluimos que  $(x)y$  puede derivarse a partir de  $S$ .

Si  $G$  una gramática regular podemos obtener un autómata  $M$  tal que  $L(M) = L(G)$ , y viceversa, dado el autómata  $M$  podemos construir la gramática  $G$ .

Mostramos un método simple y ejemplos:

**Parte 1:** Dada una gramática  $G$  mostraremos como construir un autómata  $M$  tal que  $L(G) = L(M)$ .

**Parte 2:** Dado un autómata  $M$  mostraremos como construir una gramática  $G$  tal que  $L(M) = L(G)$

**Parte 1:** Dada  $G=(V_N, V_T, S, P)$  construimos  
 $M = (K, V_T, \delta, S, F)$  donde  $K=V_N \cup \{A\}$  con A nuevo (no perteneciente a  $V_N$ )

Si  $P$  tiene la producción  $S \rightarrow \lambda$  entonces  
 $F = \{A, S\}$  si no  $F = \{A\}$ .

Al definir  $\delta$  debemos considerar los siguientes casos

- si  $B \rightarrow a \in P$  entonces:  
 $\delta(B, a) \supseteq \{A\},$   
 $a \in V_T; B \in V_N$
- si  $B \rightarrow aC \in P$  entonces:  
 $\delta(B, a) \supseteq \{C\},$   
 $a \in V_T; B, C \in V_N$

## Ejemplo

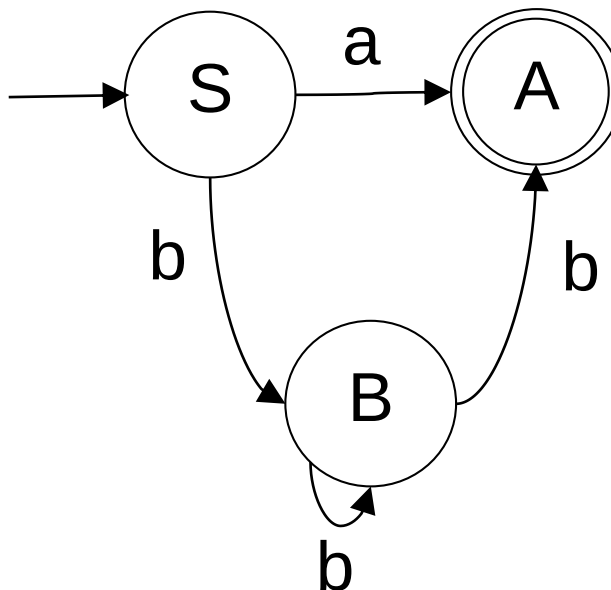
$$G = (V_N, V_T, S, P)$$

$$V_N = \{S, B\}$$

$$V_T = \{a, b\}$$

$$S \rightarrow a \mid bB$$

$$B \rightarrow bB \mid b$$



$$M = (K, V_T, \delta, S, F)$$

$$K = \{S, A, B\}$$

$$F = \{A\}$$

$$\delta(S, a) = \{A\}$$

$$\delta(S, b) = \{B\}$$

$$\delta(B, b) = \{A, B\}$$



**Parte 2:** Dado  $M = (S, \Sigma, \delta, s_0, F)$  un AF determinista construimos

$G = (S, \Sigma, s_0, P)$  donde  $P$  está formado por

$s_0 \rightarrow \lambda$ , si  $s_0 \in F$

$B \rightarrow aC$ , si  $\delta(B, a) = C$

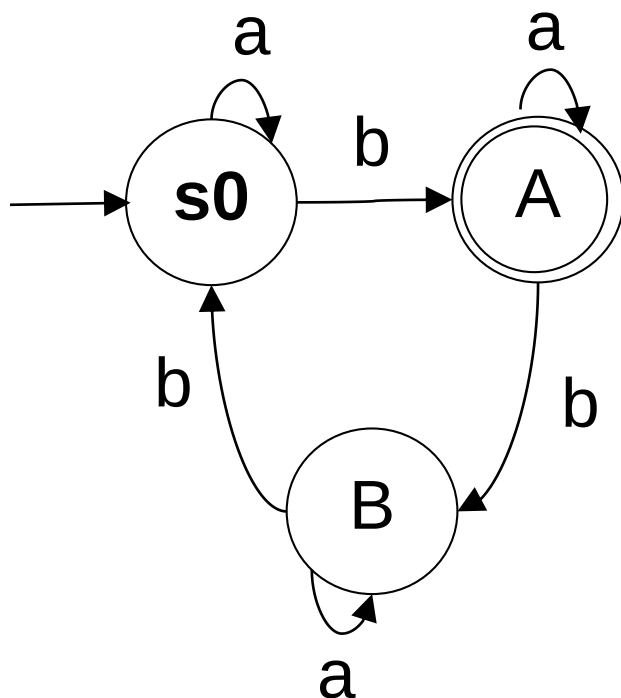
$B \rightarrow a$ , si  $\delta(B, a) = C$  y  $C \in F$

## Ejemplo

$$M = (S, \Sigma, \delta, s_0, F)$$

$$\Sigma = \{a, b\}$$

$$F = \{A\}$$



$$G = (S, \Sigma, s_0, P)$$

P:

$$s_0 \rightarrow as_0 \mid bA \mid b$$

$$A \rightarrow aA \mid bB \mid a$$

$$B \rightarrow bs_0 \mid aB$$